Pushing code to GIT Repository

In this practice you will install GIT Bash on Windows, create a repository in http://github.com and push the python application to git.

Stages:

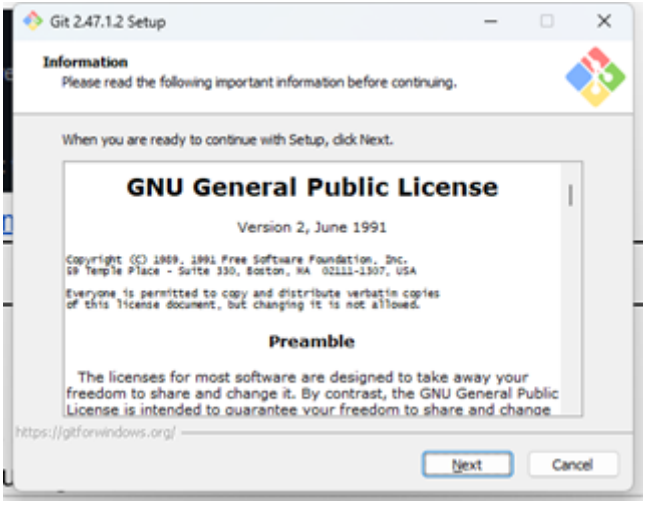
1. Install GIT Bash on windows. Mac has the 'git' by default
2. Push the python application code to GIT repository

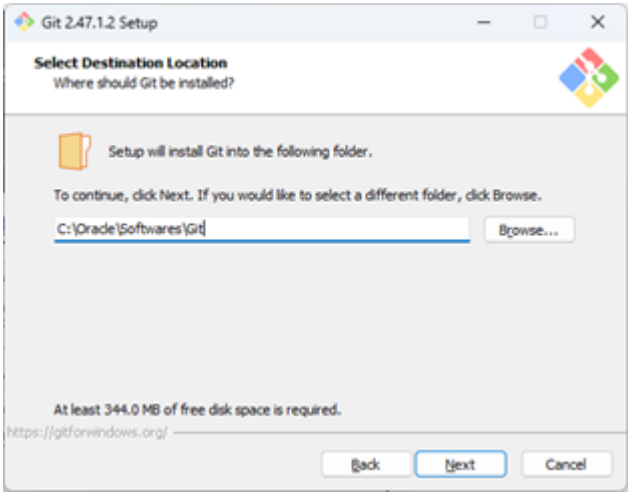
Stage A: Verify and Install GIT

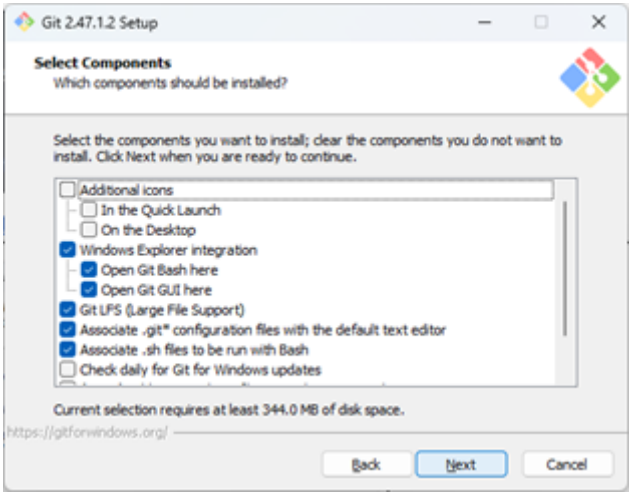
1. Verify if 'GIT' is already installed on your machine
   a. Launch a command prompt on windows / terminal on Mac and execute below command

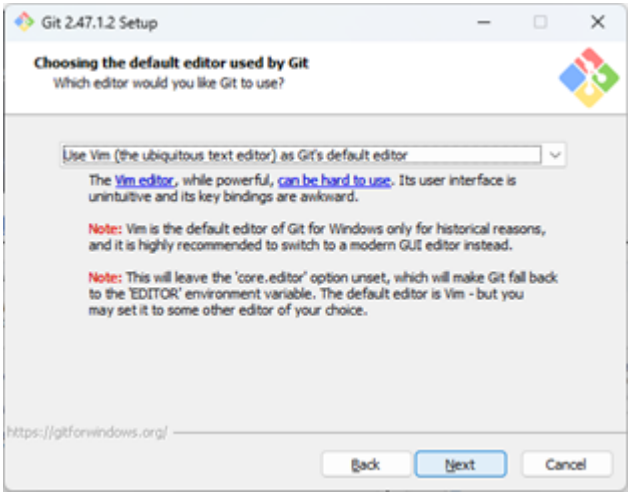
      ```
      C:\Users\syedh>git --version
      ```

   b. If the command is recognized, the installation step can be skipped.
2. Installing GIT
   a. Download and install GIT client https://git-scm.com/downloads/win
   b. Answer the wizard as below









Git 2.47.1.2 Setup

Adjusting the name of the initial branch in new repositories
What would you like Git to name the initial branch after "git init"?

- (●) **Let Git decide**
  Let Git use its default branch name (currently: "master") for the initial branch in newly created repositories. The Git project intends to change this default to a more inclusive name in the near future.
- ( ) **Override the default branch name for new repositories**
  NEW! Many teams already renamed their default branches; common choices are "main", "trunk" and "development". Specify the name "git init" should use for the initial branch:
  main

This setting does not affect existing repositories.

https://gitforwindows.org/

Back | Next | Cancel

Git 2.47.1.2 Setup

Adjusting your PATH environment
How would you like to use Git from the command line?

- ( ) **Use Git from Git Bash only**
  This is the most cautious choice as your PATH will not be modified at all. You will only be able to use the Git command line tools from Git Bash.
- (●) **Git from the command line and also from 3rd-party software**
  (Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows PowerShell as well as any third-party software looking for Git in PATH.
- ( ) **Use Git and optional Unix tools from the Command Prompt**
  Both Git and the optional Unix tools will be added to your PATH.
  Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.

https://gitforwindows.org/

Back | Next | Cancel

Git 2.47.1.2 Setup

Choosing HTTPS transport backend
Which SSL/TLS library would you like Git to use for HTTPS connections?

- (●) **Use the OpenSSL library**
  Server certificates will be validated using the ca-bundle.crt file.
- ( ) **Use the native Windows Secure Channel library**
  Server certificates will be validated using Windows Certificate Stores. This option also allows you to use your company's internal Root CA certificates distributed e.g. via Active Directory Domain Services.

https://gitforwindows.org/

Back | Next | Cancel

Git 2.47.1.2 Setup

Configuring the line ending conversions
How should Git treat line endings in text files?

- (●) **Checkout Windows-style, commit Unix-style line endings**
  Git will convert LF to CRLF when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects, this is the recommended setting on Windows ("core.autocrlf" is set to "true").
- ( ) **Checkout as-is, commit Unix-style line endings**
  Git will not perform any conversion when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects, this is the recommended setting on Unix ("core.autocrlf" is set to "input").
- ( ) **Checkout as-is, commit as-is**
  Git will not perform any conversions when checking out or committing text files. Choosing this option is not recommended for cross-platform projects ("core.autocrlf" is set to "false").

https://gitforwindows.org/

Back | Next | Cancel

Git 2.47.1.2 Setup

Configuring the terminal emulator to use with Git Bash
Which terminal emulator do you want to use with your Git Bash?

- (●) **Use MinTTY (the default terminal of MSYS2)**
  Git Bash will use MinTTY as terminal emulator, which sports a resizable window, non-rectangular selections and a Unicode font. Windows console programs (such as interactive Python) must be launched via `winpty` to work in MinTTY.
- ( ) **Use Windows' default console window**
  Git will use the default console window of Windows ("cmd.exe"), which works well with Win32 console programs such as interactive Python or node.js, but has a very limited default scroll-back, needs to be configured to use a Unicode font in order to display non-ASCII characters correctly, and prior to Windows 10 its window was not freely resizable and it only allowed rectangular text selections.

https://gitforwindows.org/

Back | Next | Cancel

Git 2.47.1.2 Setup

Choose a credential helper
Which credential helper should be configured?

- ( ) **Git Credential Manager**
  Use the cross-platform Git Credential Manager.
  See more information about the future of Git Credential Manager here.
- (●) **None**
  Do not use a credential helper.

https://gitforwindows.org/

Back | Next | Cancel

Git 2.47.1.2 Setup

Configuring extra options
Which features would you like to enable?

- [x] **Enable file system caching**
  File system data will be read in bulk and cached in memory for certain operations ("core.fscache" is set to "true"). This provides a significant performance boost.
- [ ] **Enable symbolic links**
  Enable symbolic links (requires the SeCreateSymbolicLink permission). Please note that existing repositories are unaffected by this setting.

https://gitforwindows.org/

Back | Install | Cancel

Git 2.47.1.2 Setup

Completing the Git Setup Wizard

Setup has finished installing Git on your computer. The application may be launched by selecting the installed shortcuts.

Click Finish to exit Setup.

- [ ] Launch Git Bash
- [x] View Release Notes

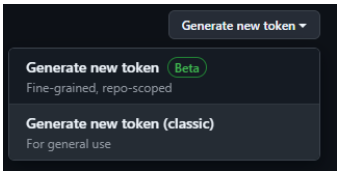
Finish

Stage B - Push python application code to git repository

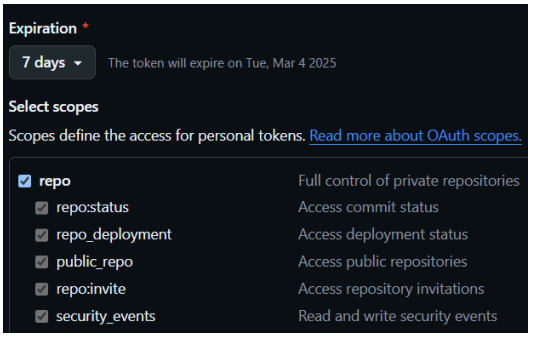
Phase 1 - Create a repository in github.com

1. Access http://github.com → Create an account using your personal email and remember the 'username & password' for it.
2. Access http://github.com → Create the repository by clicking 'New' button on left top corner to github.com

3. Mention repository name as 'ai-devops' → Scroll down and click 'Create Repository'
4. Generate the 'Personal Access Token' to be used as password while authentication.
   a. Browse the link https://github.com/settings/tokens
   b. Click on 'Generate New Token' → Click on 'Generate new token (classic)'



   c. Provide the parameters as below → Scroll down and click 'Create Token'

Note → Accessing ai-devops repository Expiration → 7 Days repo → Checked	

   d. Copy the token and keep it handy

Phase 2 - Push the code GIT Repository

1. Launch a command prompt if not started
   `cd c:\labs\ai-devops`
2. Execute the command `'git init'`
   This creates the local repository folder called '.git' in c:\Oracle\labs folder
3. Provide your identity using below commands (user name & email should the one's used in github.com)
   `git config --global user.email email-used-for-git@domain.com`
   `git config --global user.name your-git-user-name`
4. Execute bellow commands to add & commit code to local repository
   `git add sample-python-repo`
   `git commit -m "first commit message for application"`
5. Execute below commands
   `git branch -M main`
   `git remote add origin https://github.com/`**`yourUserName`**`/ai-devops.git`
   `git push -u origin main`
   a. Click on 'Token' tab → Enter the token created in Phase 1 → Click Sign In
6. To verify access the repository URL https://github.com/yourUserName/ai-devops.git